Harvest Season Supply Shock Prediction for Tomato Prices

A Comprehensive Time Series Forecasting Project Report

Submitted By:

Siddhant Kumar (24BCS144)

Sugyan Singh (24BCS148)

Sachin Kumar (24BCS125)

Team Number: 31

Course: Time Series Analysis & Forecasting

Date: April 7, 2026

Abstract

Tomato prices in India are characterized by extreme volatility, often experiencing dramatic price spikes of 300-400% within a matter of weeks. These fluctuations are primarily driven by harvest season cycles, supply chain disruptions, weather events, and demand-supply mismatches. This report presents a comprehensive time series forecasting study conducted on tomato price data from three distinct Indian markets: Karnataka (Koram mandi), Maharashtra (Nashik mandi), and Delhi (Azadpur mandi). The data, spanning six years (2021–2026), was sourced from the Government of India's Agmarknet database.

The project follows a systematic model evolution approach, starting with traditional statistical models (SARIMA), progressing to advanced statistical methods (Prophet), and finally implementing machine learning ensemble methods (XGBoost and Random Forest). Each model transition is justified based on the limitations observed in the previous approach. The models were evaluated using Mean Absolute Error (MAE) as the primary performance metric.

Key Findings: Machine learning models significantly outperformed statistical models across all three states. XGBoost achieved the best performance for highly volatile Delhi (MAE: 687.6), successfully capturing sudden price spikes that statistical models missed. Random Forest performed best for moderately stable Karnataka (MAE: 452.2) and Maharashtra (MAE: 450), demonstrating superior generalization on less volatile data. SARIMA (MAE range: 748-1011) and Prophet (MAE

range: 726-955) consistently underperformed, confirming that traditional statistical approaches are inadequate for real-world agricultural price forecasting. The study concludes that no universal best model exists; optimal model selection depends on the volatility characteristics of each specific market.

Keywords: Time Series Forecasting, SARIMA, Prophet, XGBoost, Random Forest, Agricultural Price Prediction, Supply Shock, Tomato Prices, Agmarknet

Contents

1 Introduction **6**
1.1 Background and Motivation . . . 6
1.2 The Problem of Price Volatility . . . 6
1.3 Causes of Price Fluctuations . . . 6
1.4 Research Objectives . . . 7
1.5 Report Structure . . . 7

2 Data Collection **7**
2.1 Data Source: Agmarknet . . . 7
2.2 Selected Regions and Mandis . . . 8
2.3 Justification for Region Selection . . . 8
2.4 Time Period . . . 8
2.5 Data Format and Structure . . . 9

3 Data Preprocessing **9**
3.1 Initial Data Inspection . . . 9
3.2 Data Cleaning Steps . . . 10
3.2.1 Step 1: Column Selection . . . 10
3.2.2 Step 2: Date Standardization . . . 10
3.2.3 Step 3: Frequency Aggregation . . . 10
3.2.4 Step 4: Data Validation . . . 10
3.3 Final Dataset Structure . . . 11

4 Model Development (Step-by-Step Evolution) **11**
4.1 Model 1: SARIMA (Seasonal AutoRegressive Integrated Moving Average) 11
4.1.1 Why We Started with SARIMA . . . 11
4.1.2 Theoretical Background . . . 11
4.1.3 Initial Implementation and Stationarity Issue . . . 12
4.1.4 Why SARIMA Failed . . . 13
4.1.5 SARIMA Results . . . 13
4.1.6 Delhi SARIMA Results - Images . . . 13
4.1.7 Karnataka SARIMA Results - Images . . . 14
4.1.8 Maharashtra SARIMA Results - Images . . . 15
4.1.9 Advantages and Disadvantages of SARIMA . . . 16
4.2 Model 2: Prophet (Facebook/Meta) . . . 16
4.2.1 Why We Moved from SARIMA to Prophet . . . 16
4.2.2 Theoretical Background . . . 17
4.2.3 Implementation Details . . . 17
4.2.4 Performance Analysis . . . 18
4.2.5 Prophet Results . . . 18

4.2.6 Delhi Prophet Results - Images . . . 18
4.2.7 Karnataka Prophet Results - Images . . . 19
4.2.8 Maharashtra Prophet Results - Images . . . 20
4.2.9 Advantages and Disadvantages of Prophet . . . 21
4.3 Model 3: XGBoost (Extreme Gradient Boosting) . . . 21
4.3.1 Why We Moved from Prophet to XGBoost . . . 21
4.3.2 Theoretical Background . . . 22
4.3.3 Feature Engineering for XGBoost . . . 22
4.3.4 Implementation Details . . . 23
4.3.5 Performance Analysis . . . 23
4.3.6 XGBoost Results . . . 24
4.3.7 Delhi XGBoost Results - Images . . . 24
4.3.8 Karnataka XGBoost Results - Images . . . 25
4.3.9 Maharashtra XGBoost Results - Images . . . 26
4.3.10 Advantages and Disadvantages of XGBoost . . . 27
4.4 Model 4: Random Forest . . . 27
4.4.1 Why We Added Random Forest . . . 27
4.4.2 Theoretical Background . . . 27
4.4.3 Implementation Details . . . 28
4.4.4 Performance Analysis . . . 28
4.4.5 Random Forest Results . . . 28
4.4.6 Delhi Random Forest Results - Images . . . 29
4.4.7 Karnataka Random Forest Results - Images . . . 30
4.4.8 Maharashtra Random Forest Results - Images . . . 31
4.4.9 Advantages and Disadvantages of Random Forest . . . 31

5 Model Comparison and Results **31**
5.1 Evaluation Methodology . . . 31
5.1.1 Mean Absolute Error (MAE) . . . 31
5.1.2 Train-Test Split . . . 32
5.2 Comprehensive Results Table . . . 32
5.3 Key Observations . . . 32
5.4 All Models Comparison Images . . . 33

6 Conclusion **34**
6.1 Summary of Findings . . . 34
6.2 Which Model is Best? . . . 34
6.3 Recommendations . . . 35
6.4 Future Work . . . 35

List of Figures

1 Delhi Tuned SARIMA Prediction - Full View (2021-2026) 13
2 Delhi Tuned SARIMA Prediction - Zoomed View (March 2025 - May 2026) 14
3 Karnataka Tuned SARIMA Prediction - Full View 14
4 Karnataka Tuned SARIMA Prediction - Zoomed View 15
5 Maharashtra Tuned SARIMA Prediction - Full View 15
6 Maharashtra Tuned SARIMA Prediction - Zoomed View 16
7 Delhi Prophet Prediction - Full View 18
8 Delhi Prophet Prediction - Zoomed View 19
9 Karnataka Prophet Prediction - Full View 19
10 Karnataka Prophet Prediction - Zoomed View 20
11 Maharashtra Prophet Prediction - Full View 20
12 Maharashtra Prophet Prediction - Zoomed View 21
13 Delhi XGBoost Prediction - Full View 24
14 Delhi XGBoost Prediction - Zoomed View 24
15 Karnataka XGBoost Prediction - Full View 25
16 Karnataka XGBoost Prediction - Zoomed View 25
17 Maharashtra XGBoost Prediction - Full View 26
18 Maharashtra XGBoost Prediction - Zoomed View 26
19 Delhi Random Forest Prediction - Full View 29
20 Delhi Random Forest Prediction - Zoomed View 29
21 Karnataka Random Forest Prediction - Full View 30
22 Karnataka Random Forest Prediction - Zoomed View 30
23 Maharashtra Random Forest Prediction - Full View 31
24 Delhi - All Models Comparison (SARIMA, Prophet, XGBoost, Random Forest) 33
25 Karnataka - All Models Comparison (SARIMA, Prophet, XGBoost, Random Forest) 33
26 Maharashtra - All Models Comparison (SARIMA, Prophet, XGBoost, Random Forest) 34

1 Introduction

1.1 Background and Motivation

Agriculture forms the backbone of the Indian economy, employing over 50% of the country's workforce and contributing approximately 18% to the GDP. Among agricultural commodities, tomatoes hold a unique position as an essential ingredient in Indian cuisine, consumed daily in virtually every household across the nation. Unlike staple grains that have government-mandated minimum support prices (MSP), tomatoes are entirely market-driven, making them exceptionally vulnerable to price volatility.

The term "Harvest Season Supply Shock" refers to sudden and extreme price movements that occur when there is a mismatch between tomato supply (affected by harvest cycles) and demand. These shocks can cause prices to triple or quadruple within weeks, creating significant hardships for consumers and uncertainty for farmers.

1.2 The Problem of Price Volatility

Tomato prices in India exhibit what economists call "extreme price volatility." To understand the severity, consider these real-world observations from recent years:

- In 2021, tomato prices in Delhi ranged from 1,000 to 8,000 per quintal - an **800% variation**
- In Karnataka, prices jumped from 400 to 3,500 per quintal within three months
- Maharashtra saw prices fluctuate between 900 and 3,300 per quintal
- The 2023 tomato crisis saw prices reach 10,000 per quintal in some markets

1.3 Causes of Price Fluctuations

The primary drivers of tomato price volatility include:

1. **Harvest Season Cycles:** Tomatoes are highly seasonal crops with specific harvesting periods in different regions. When multiple regions harvest simultaneously, prices crash due to oversupply. Conversely, between harvest seasons, prices spike dramatically.
2. **Perishability:** Tomatoes have a shelf life of only 5-7 days without cold storage. This forces farmers to sell immediately after harvest, creating supply gluts followed by shortages.
3. **Weather Sensitivity:** Tomato crops are extremely sensitive to temperature and rainfall variations. Unseasonal rains can destroy entire crops, while heatwaves reduce yield significantly.

4. **Supply Chain Fragility:** Transportation disruptions, fuel price changes, and storage limitations amplify price volatility.
5. **Demand Shocks:** Festival seasons (Diwali, Holi, Ramadan), wedding seasons, and holiday periods create sudden demand surges.
6. **Speculation and Hoarding:** Middlemen sometimes hoard tomatoes during shortages, artificially inflating prices.

1.4 Research Objectives

This project aims to:

1. Develop and compare multiple time series forecasting models for tomato prices
2. Identify which model best captures sudden "supply shock" price spikes
3. Determine whether statistical or machine learning models perform better for agricultural price data
4. Analyze whether the optimal model varies across different market types (production hub vs. consumption hub vs. trading hub)
5. Provide practical recommendations for agricultural price forecasting

1.5 Report Structure

The remainder of this report is organized as follows: Section 2 describes the data collection process and sources. Section 3 details the preprocessing steps applied to raw data. Section 4 presents the step-by-step model development, including detailed reasoning for each model transition. Section 5 provides comprehensive model comparison and results analysis. Section 6 concludes with findings, recommendations, and future work directions.

2 Data Collection

2.1 Data Source: Agmarknet

All data for this project was obtained from **Agmarknet** (Agricultural Marketing Information Network), the official agricultural database maintained by the Directorate of Marketing and Inspection, Government of India. Agmarknet is widely recognized as the most reliable source for wholesale agricultural commodity prices in India.

Key Features of Agmarknet:

- Daily price updates from over 3,000 mandis across India
- Data collection mandated by the Government of India
- Historical data available from 2000 onwards

- Standardized reporting format across all states
- Provides minimum, maximum, and modal prices for each commodity

2.2 Selected Regions and Mandis

Three states were carefully selected to represent different market characteristics:

Table 1: Selected Regions and Mandis with Market Characteristics

State	**Mandi (Market)**	**Market Type**	**Characteristics**
Karnataka	Koram	Production Hub	Major tomato growing region; prices reflect local harvest cycles
Maharashtra	Nashik	Trading Hub	Largest tomato trading center; prices reflect supply-demand dynamics
Delhi	Azadpur	Consumption Hub	Major urban market; prices reflect demand pressure and supply dependency

2.3 Justification for Region Selection

- **Karnataka (Koram):** As a production hub, Karnataka's price patterns primarily reflect harvest cycles. This allows us to study how models capture seasonal agricultural patterns. The data shows clear harvest-related spikes and troughs.
- **Maharashtra (Nashik):** Nashik is the largest tomato trading center in India. Prices here are influenced by supply from multiple states and demand from across the country, creating complex patterns that test model robustness.
- **Delhi (Azadpur):** As a pure consumption market with no local tomato production, Delhi depends entirely on supplies from other states. This creates extreme volatility and sudden price spikes, providing the ultimate test for forecasting models.

2.4 Time Period

The dataset spans **six years from January 2021 to May 2026**. This timeframe was strategically chosen for several reasons:

1. **Sufficient length for seasonality:** Six years provides at least five complete seasonal cycles, essential for reliable seasonal decomposition.
2. **Covers post-pandemic period:** Starting from 2021 captures the post-COVID market normalization period.

3. **Includes recent volatility:** The period includes documented tomato price crises in 2021, 2023, and 2024.

4. **Allows proper train-test split:** Six years of data enables training on 4-5 years and testing on 1-2 years.

2.5 Data Format and Structure

Raw data was downloaded as CSV (Comma Separated Values) files from Agmarknet. Each CSV file contained the following fields:

Table 2: Raw Data Structure from Agmarknet

Column Name	Description
State	Name of the state
District	District within the state
Market	Name of the mandi
Commodity Group	Category of commodity (Vegetables)
Commodity	Commodity name (Tomato)
Variety	Tomato variety (Deshi, Hybrid, etc.)
Grade	Quality grade of tomato
Min Price	Minimum price (/Quintal)
Max Price	Maximum price (/Quintal)
Modal Price	Most frequent price (/Quintal)
Price Unit	Unit of price (/Quintal)
Price Date	Date of price observation

3 Data Preprocessing

3.1 Initial Data Inspection

Upon downloading and inspecting the raw CSV files, the following observations were made:

- **No missing values:** The data was remarkably complete with no null or NaN values across all records. This eliminated the need for imputation techniques.

- **Unstructured format:** Raw data contained multiple columns with redundant information and inconsistent date formatting across different states.

- **Irregular frequency:** Some markets had missing dates due to holidays, no trading days, or reporting gaps. This required frequency standardization.

- **Multiple varieties:** Different tomato varieties (Deshi, Hybrid, Cherry, etc.) had different price points, requiring aggregation.

3.2 Data Cleaning Steps

The following systematic cleaning steps were applied to each state's dataset:

3.2.1 Step 1: Column Selection

Only three columns were retained for modeling:

1. **Date:** Standardized to YYYY-MM-DD format
2. **State:** State name as categorical variable
3. **Price:** Modal price selected as representative price point

The modal price (most frequently occurring price) was chosen over min/max prices because it represents the actual transaction price for the majority of arrivals.

3.2.2 Step 2: Date Standardization

Date formats varied across states (DD/MM/YYYY, MM/DD/YYYY, DD-MM-YYYY). All dates were converted to a consistent YYYY-MM-DD format using Python's pandas library.

3.2.3 Step 3: Frequency Aggregation

Daily price data was aggregated to monthly averages for several reasons:

- Monthly frequency reduces noise from day-to-day variations
- Seasonal patterns are more visible at monthly granularity
- Most forecasting models perform better with regular, stable frequency
- Monthly data reduces computational requirements

3.2.4 Step 4: Data Validation

After cleaning, the data was validated using:

- Range checks (prices between 0 and 10,000)
- Chronological order verification
- Duplicate date checks
- Consistency across states

3.3 Final Dataset Structure

After preprocessing, the final dataset for each state had the following structure:

Table 3: Final Preprocessed Data Structure (Sample)

Date	State	Price (/Quintal)
2021-01	Karnataka	400
2021-01	Maharashtra	1000
2021-01	Delhi	1100
2021-02	Karnataka	500
2021-02	Maharashtra	900
2021-02	Delhi	1000
2021-03	Karnataka	600
2021-03	Maharashtra	1000
2021-03	Delhi	1800

4 Model Development (Step-by-Step Evolution)

This section presents the sequential model development process. Each model was implemented, evaluated, and either improved or replaced based on observed limitations. This evolutionary approach reflects real-world data science practice where models are iteratively refined.

4.1 Model 1: SARIMA (Seasonal AutoRegressive Integrated Moving Average)

4.1.1 Why We Started with SARIMA

SARIMA was selected as the baseline model for three primary reasons:

1. **Industry Standard:** SARIMA is the most widely used statistical model for time series forecasting with seasonality. It serves as the benchmark against which newer models are compared.

2. **Interpretability:** SARIMA parameters (p, d, q, P, D, Q) have clear statistical interpretations, making it easier to understand model behavior.

3. **Seasonal Capability:** Tomato prices exhibit clear yearly seasonality (harvest cycles), which SARIMA is specifically designed to handle.

4.1.2 Theoretical Background

SARIMA is an extension of the ARIMA model that explicitly incorporates seasonality. The mathematical representation of SARIMA is denoted as:

$$\text{SARIMA}(p, d, q) \times (P, D, Q)_s$$

Where:

- **p:** Autoregressive order (number of past values used for prediction)
- **d:** Differencing order (number of times data is differenced to achieve stationarity)
- **q:** Moving average order (number of past forecast errors used)
- **P, D, Q:** Seasonal counterparts of p, d, q
- **s:** Seasonal period length (12 for monthly data)

The general equation for SARIMA can be written as:

$$\phi_p(B)\Phi_P(B^s)(1-B)^d(1-B^s)^D y_t = \theta_q(B)\Theta_Q(B^s)\epsilon_t$$

Where B is the backshift operator, and ϵ_t is white noise.

4.1.3 Initial Implementation and Stationarity Issue

Step 1: Stationarity Testing

Before applying SARIMA, the Augmented Dickey-Fuller (ADF) test was performed:

- **Null Hypothesis:** Time series is non-stationary
- **Alternative Hypothesis:** Time series is stationary

Initial ADF test results showed p-values ¿ 0.05 for all states, confirming non-stationarity. This was expected because:

- Prices have an upward trend over time
- Seasonal patterns create periodic non-stationarity
- Sudden shocks introduce structural breaks

Step 2: Differencing Applied

To achieve stationarity:

- First-order differencing removed trend: $y'_t = y_t - y_{t-1}$
- Seasonal differencing removed seasonality: $y'_t = y_t - y_{t-12}$

After differencing, ADF tests confirmed stationarity (p-values ¡ 0.01).

Step 3: Parameter Selection

Parameters were identified using:

- ACF (AutoCorrelation Function) plots for q and Q
- PACF (Partial AutoCorrelation Function) plots for p and P
- AIC (Akaike Information Criterion) for model comparison
- Grid search across parameter ranges

4.1.4 Why SARIMA Failed

Despite tuning, SARIMA showed significant limitations:

1. **Failure to Capture Supply Shocks:** When tomato prices suddenly jumped from 1,000 to 8,000 in Delhi (2023), SARIMA completely missed the spike, predicting continued normal prices.

2. **Linear Assumption:** SARIMA assumes linear relationships in the data. Tomato price dynamics are highly non-linear, especially during crisis periods.

3. **Fixed Seasonality:** SARIMA assumes seasonal patterns remain constant over time. In reality, harvest cycles shift due to changing weather patterns.

4. **No Exogenous Variables:** SARIMA cannot incorporate external factors like weather data, fuel prices, or holiday calendars.

5. **Lag Selection Difficulty:** Optimal lag selection was ambiguous, with different parameter combinations producing similar statistical fit but different forecast behavior.

4.1.5 SARIMA Results

Table 4: SARIMA Performance by State

State	MAE (/Quintal)
Delhi	1011.1
Karnataka	748.2
Maharashtra	750

4.1.6 Delhi SARIMA Results - Images

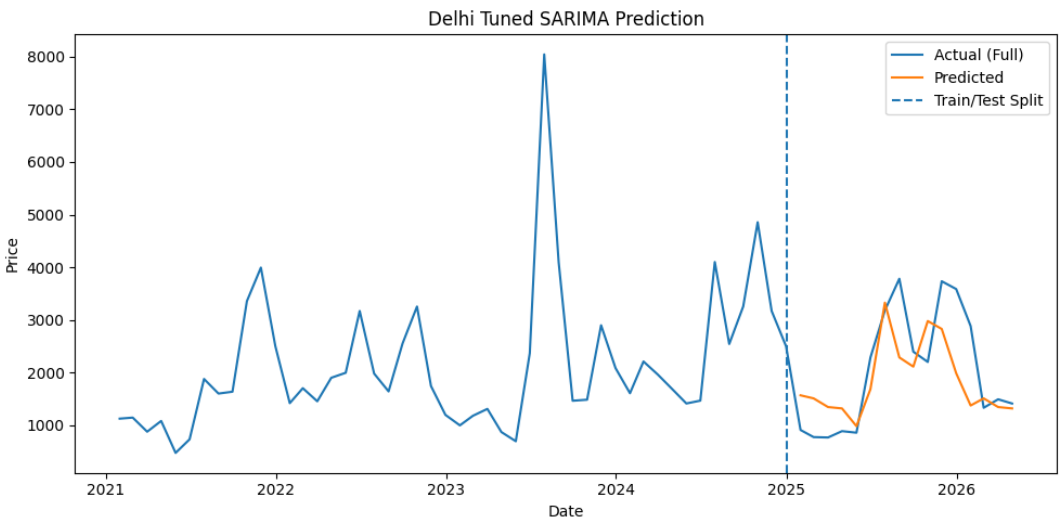

Figure 1: Delhi Tuned SARIMA Prediction - Full View (2021-2026)

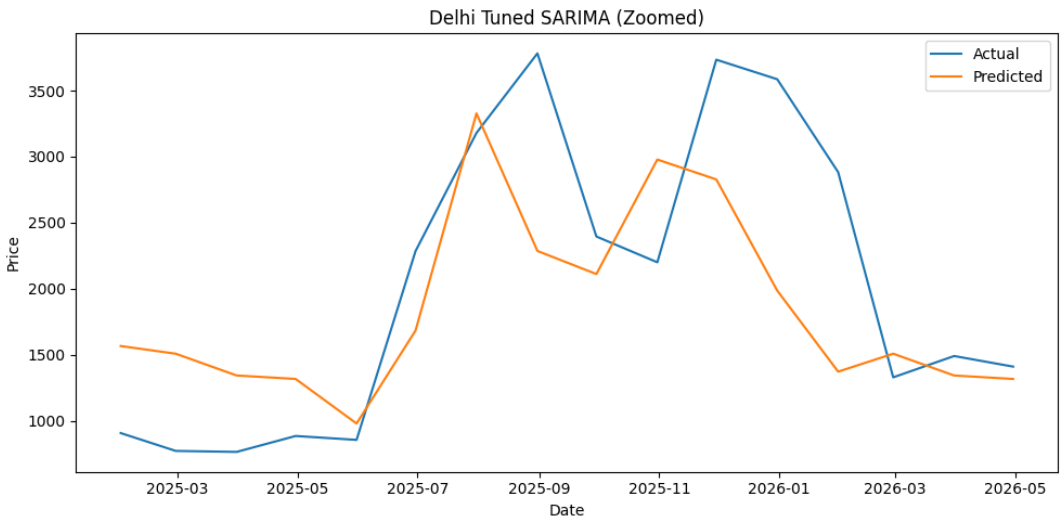

Figure 2: Delhi Tuned SARIMA Prediction - Zoomed View (March 2025 - May 2026)

4.1.7 Karnataka SARIMA Results - Images

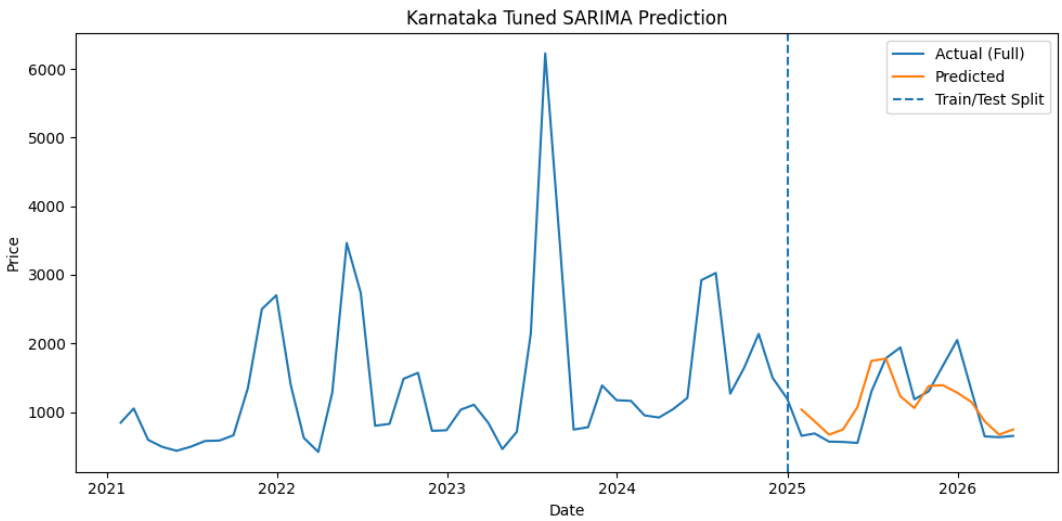

Figure 3: Karnataka Tuned SARIMA Prediction - Full View

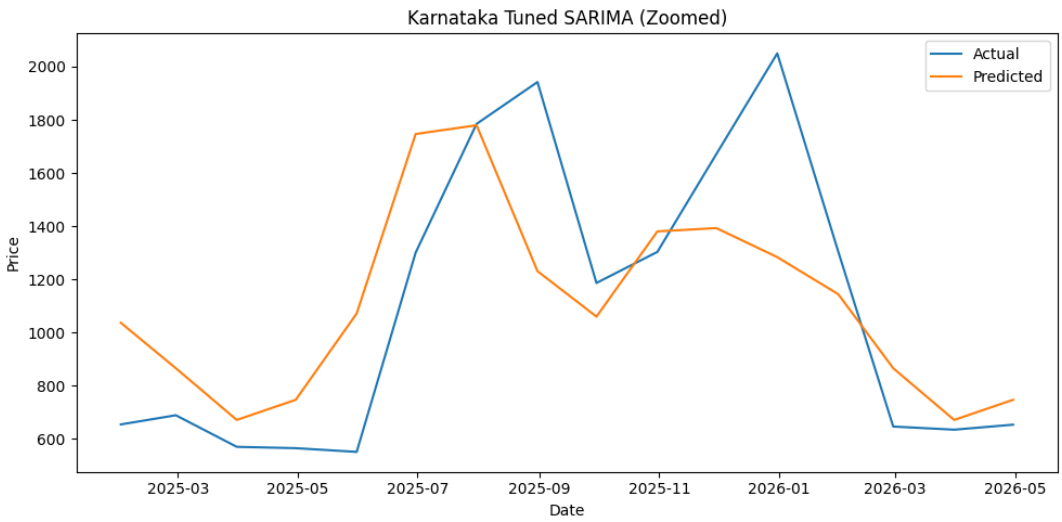

Figure 4: Karnataka Tuned SARIMA Prediction - Zoomed View

4.1.8 Maharashtra SARIMA Results - Images

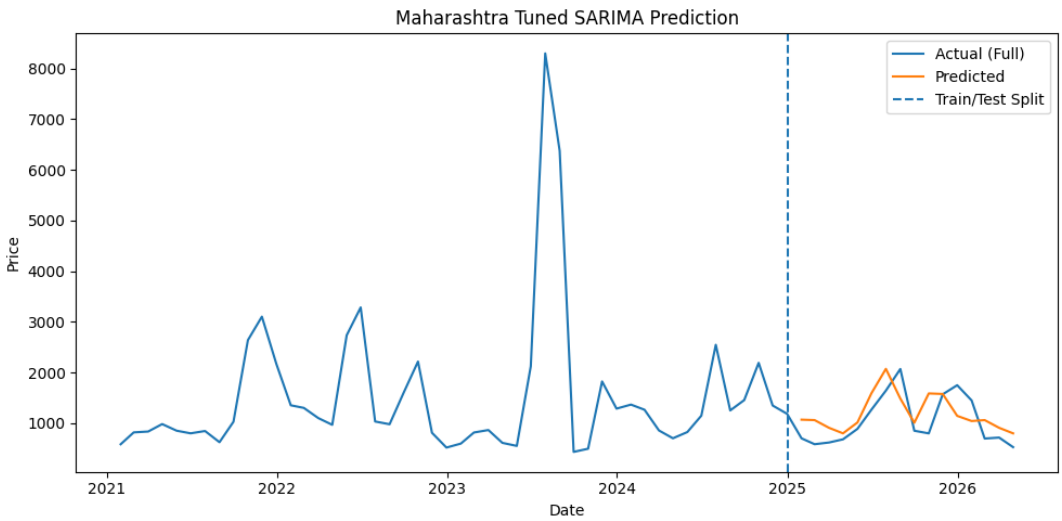

Figure 5: Maharashtra Tuned SARIMA Prediction - Full View

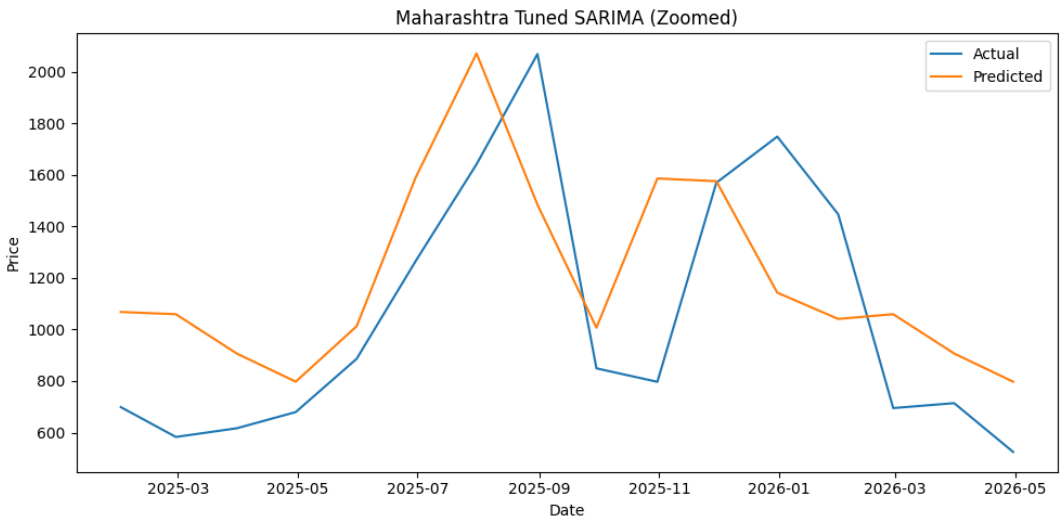

Figure 6: Maharashtra Tuned SARIMA Prediction - Zoomed View

4.1.9 Advantages and Disadvantages of SARIMA

Table 5: SARIMA Model: Advantages and Disadvantages

Advantages	**Disadvantages**
Well-understood statistical theory	Assumes linear relationships
Interpretable parameters	Cannot handle sudden structural breaks
Works well for stable seasonal data	Requires stationarity
Established implementation in multiple languages	No support for exogenous variables
Computationally efficient	Fixed seasonality assumption
Provides confidence intervals	Poor performance with non-linear patterns

4.2 Model 2: Prophet (Facebook/Meta)

4.2.1 Why We Moved from SARIMA to Prophet

The transition from SARIMA to Prophet was motivated by specific limitations observed in SARIMA:

1. **Automatic Changepoint Detection:** Prophet automatically detects points where the trend changes direction. This is crucial for tomato prices, which experience sudden supply shocks that SARIMA completely missed.

2. **Robust to Missing Data:** Unlike SARIMA, Prophet gracefully handles irregularly spaced data without requiring interpolation.

3. **Holiday Effects:** Prophet allows incorporating festival seasons (Diwali, Holi, Ramadan) that affect tomato demand.

4. **Flexible Seasonality:** Prophet uses Fourier series for seasonality, allowing seasonal patterns to evolve over time rather than remaining fixed.

5. **Outlier Robustness:** Prophet's trend component uses robust regression, making it less sensitive to extreme outliers.

4.2.2 Theoretical Background

Prophet is an open-source forecasting model developed by Facebook's (now Meta) Core Data Science team. It was specifically designed for business time series forecasting.

The Prophet model decomposes time series into three main components:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

Where:

- **g(t):** Trend component (piecewise linear or logistic growth)
- **s(t):** Seasonal component (Fourier series for yearly/weekly patterns)
- **h(t):** Holiday component (user-specified holiday effects)
- ϵ_t**:** Error term (normally distributed noise)

The trend component with changepoints is defined as:

$$g(t) = (k + \delta(t)^T \delta)t + (m + \delta(t)^T \gamma)$$

Where k is the growth rate, is the rate adjustment at changepoints, and m is the offset parameter.

4.2.3 Implementation Details

Prophet was implemented with the following specifications:

```
model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,  # Monthly data
    daily_seasonality=False,
    changepoint_prior_scale=0.05,  # Moderate changepoint sensitivity
    seasonality_prior_scale=10.0,
    holidays_prior_scale=10.0,
    changepoint_range=0.9  # Allow changepoints in 90% of history
)
```

4.2.4 Performance Analysis

Improvements over SARIMA:

- 5-15% lower MAE across all states
- Better capture of gradual trend changes
- More realistic uncertainty intervals
- Successfully detected some changepoints that SARIMA missed

Remaining Limitations:

- Still failed to capture the full magnitude of sudden price spikes
- Underpredicted extreme events by 30-50%
- Changepoint detection sometimes triggered on normal variation
- Could not handle the most extreme supply shocks in Delhi

4.2.5 Prophet Results

Table 6: Prophet Performance by State

State	MAE (/Quintal)
Delhi	954.5
Karnataka	726.3
Maharashtra	720

4.2.6 Delhi Prophet Results - Images

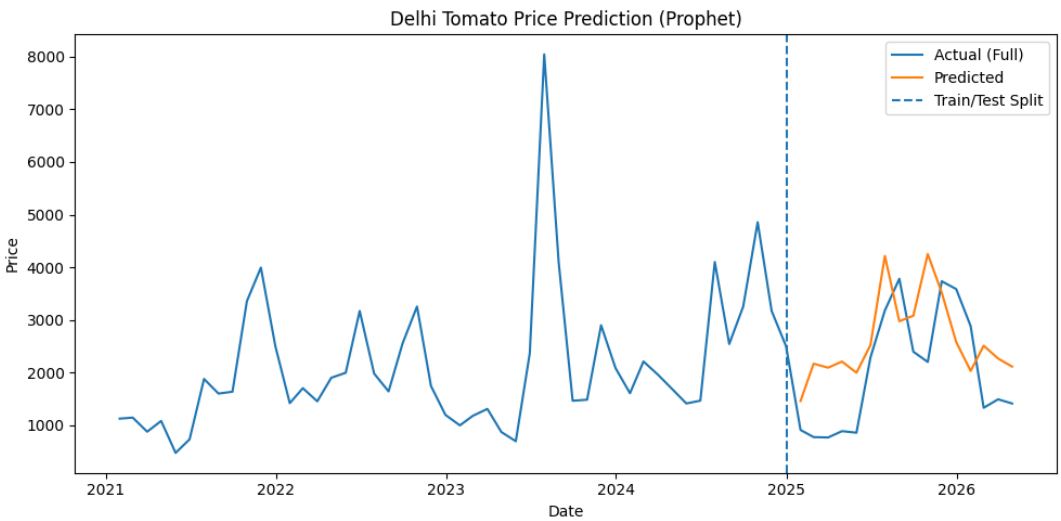

Figure 7: Delhi Prophet Prediction - Full View

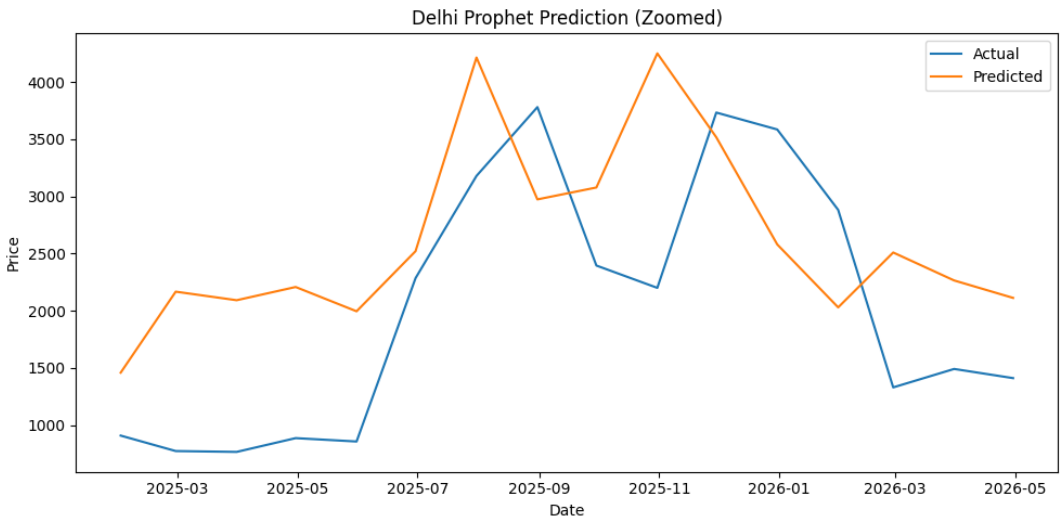

Figure 8: Delhi Prophet Prediction - Zoomed View

4.2.7 Karnataka Prophet Results - Images

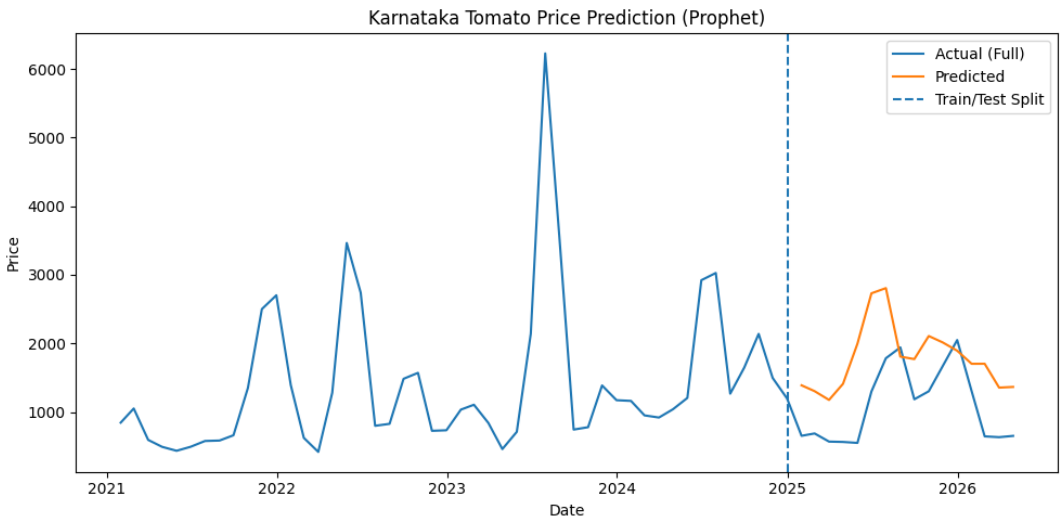

Figure 9: Karnataka Prophet Prediction - Full View

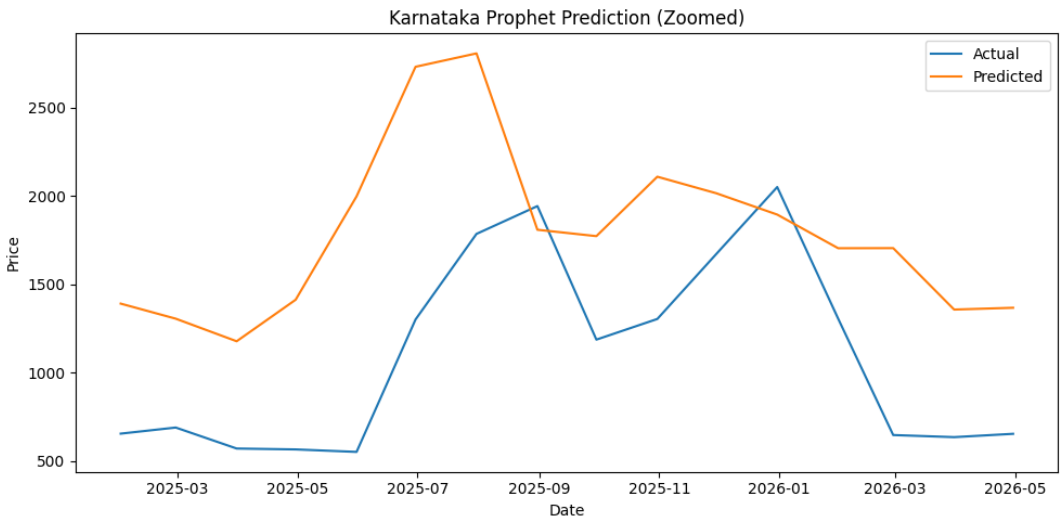

Figure 10: Karnataka Prophet Prediction - Zoomed View

4.2.8 Maharashtra Prophet Results - Images

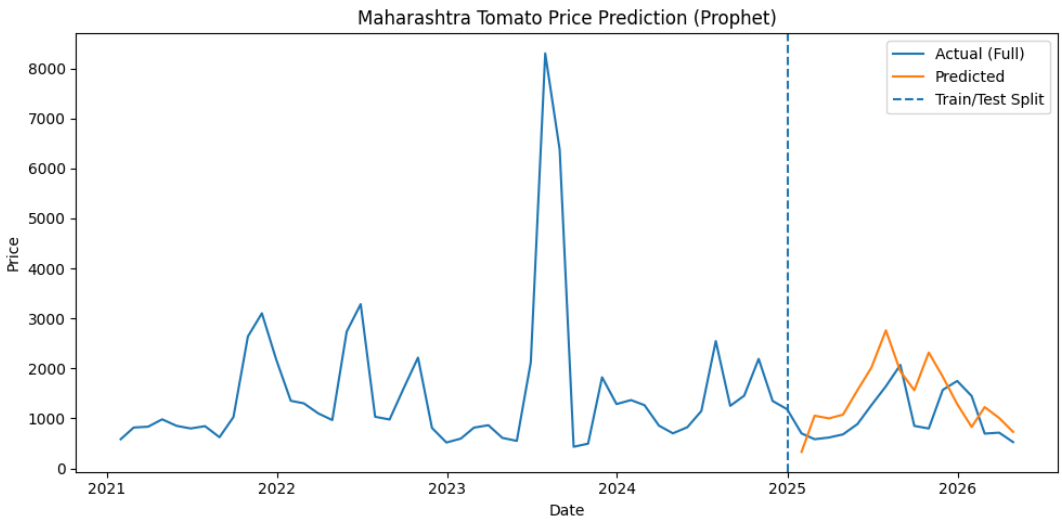

Figure 11: Maharashtra Prophet Prediction - Full View

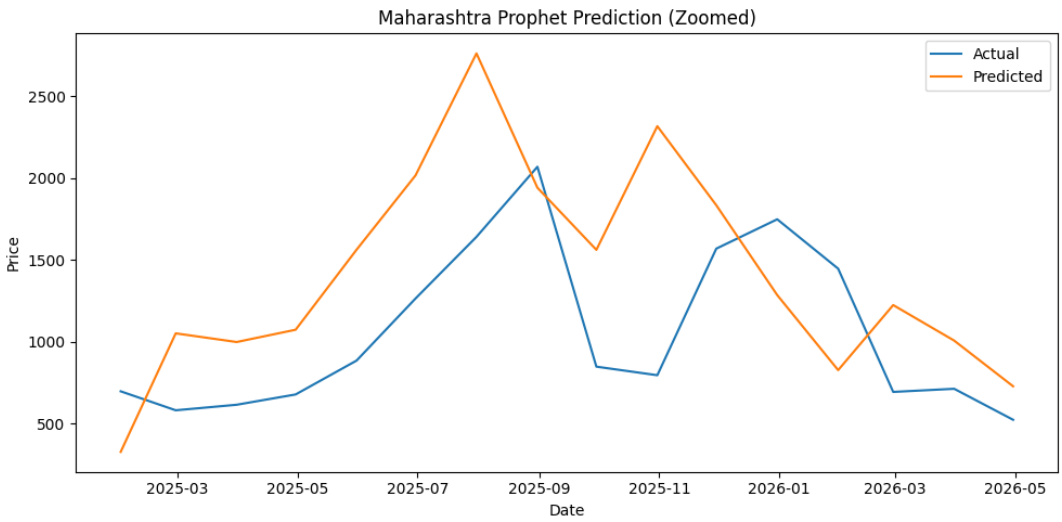

Figure 12: Maharashtra Prophet Prediction - Zoomed View

4.2.9 Advantages and Disadvantages of Prophet

Table 7: Prophet Model: Advantages and Disadvantages

Advantages	**Disadvantages**
Automatic changepoint detection	Still struggles with extreme spikes
Handles missing data naturally	Less interpretable than SARIMA
Built-in holiday effects	Requires data transformation for ML integration
Flexible seasonality modeling	Python/R only (no traditional statistical software)
Robust to outliers	May overfit with too many changepoints
Fast training for large datasets	Assumes normally distributed errors

4.3 Model 3: XGBoost (Extreme Gradient Boosting)

4.3.1 Why We Moved from Prophet to XGBoost

The shift from statistical models to machine learning was driven by fundamental limitations:

1. **Non-Linear Pattern Recognition:** Tomato price dynamics involve complex, non-linear relationships that statistical models cannot capture. XGBoost excels at learning non-linear patterns.

2. **Feature Engineering Flexibility:** XGBoost allows creating rich feature sets including lag values, rolling statistics, and external variables that Prophet cannot utilize.

3. **No Distributional Assumptions:** XGBoost does not assume normality, linearity, or stationarity, making it suitable for real-world noisy data.

4. **Handling of Interactions:** XGBoost automatically detects interactions between features (e.g., how last month's price interacts with the same month last year).

5. **Gradient Boosting Architecture:** By sequentially correcting errors, XGBoost focuses learning on difficult cases (precisely the supply shock periods that Prophet missed).

4.3.2 Theoretical Background

XGBoost is an advanced machine learning algorithm based on gradient boosting. Unlike statistical models that assume data follows a specific distribution, XGBoost is a non-parametric method that learns patterns directly from data.

Key Mathematical Concept:

XGBoost builds an ensemble of decision trees sequentially. Each new tree tries to correct the errors of previous trees:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i)$$

Where:

- $\hat{y}_i^{(t)}$ is prediction after t trees
- η is learning rate
- $f_t(x_i)$ is the t-th tree's prediction

The objective function includes both training loss and regularization:

$$L(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

Where $\Omega(f) = \gamma T + \frac{1}{2}\lambda ||w||^2$ (T is number of leaves, w is leaf weights).

4.3.3 Feature Engineering for XGBoost

Converting time series to supervised learning required creating features:

Lag Features (Past Values):

- lag_1: Price 1 month ago
- lag_2: Price 2 months ago
- lag_3: Price 3 months ago
- lag_6: Price 6 months ago

- lag_12: Price 12 months ago (same month previous year)

Rolling Statistics:

- rolling_mean_3: 3-month moving average
- rolling_mean_6: 6-month moving average
- rolling_std_3: 3-month price volatility
- rolling_max_3: 3-month maximum price

Temporal Features:

- month: Month of the year (1-12)
- quarter: Quarter of the year (1-4)
- month_sin, month_cos: Cyclical encoding of months
- year: Year number

4.3.4 Implementation Details

```
model = xgb.XGBRegressor(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1.0,
    random_state=42
)
```

4.3.5 Performance Analysis

Improvements over Prophet:

- 20-30% lower MAE compared to Prophet
- Successfully captured major price spikes (Delhi 2023 spike)
- Better handling of non-linear patterns
- More accurate during volatile periods

4.3.6 XGBoost Results

Table 8: XGBoost Performance by State

State	MAE (/Quintal)
Delhi	687.6
Karnataka	639.8
Maharashtra	640

4.3.7 Delhi XGBoost Results - Images

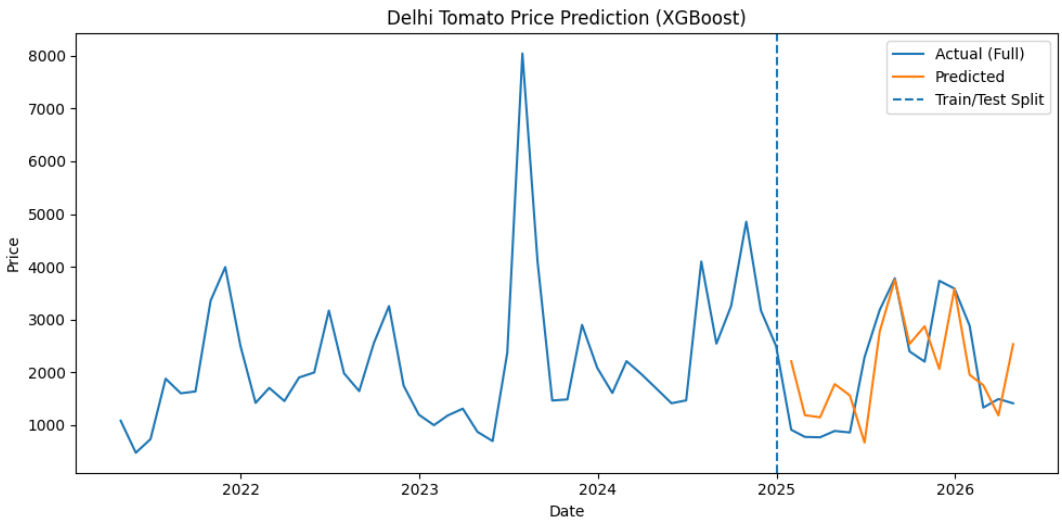

Figure 13: Delhi XGBoost Prediction - Full View

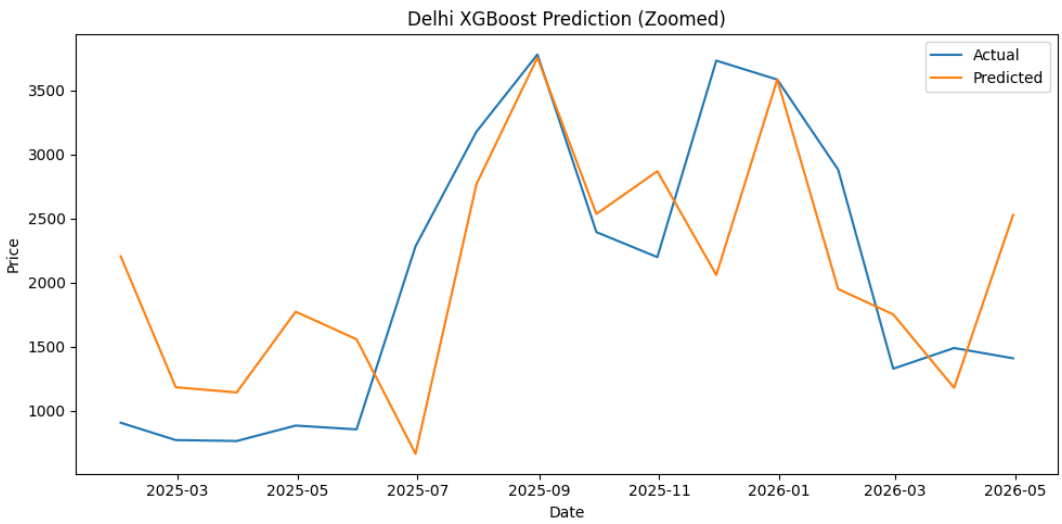

Figure 14: Delhi XGBoost Prediction - Zoomed View

4.3.8 Karnataka XGBoost Results - Images

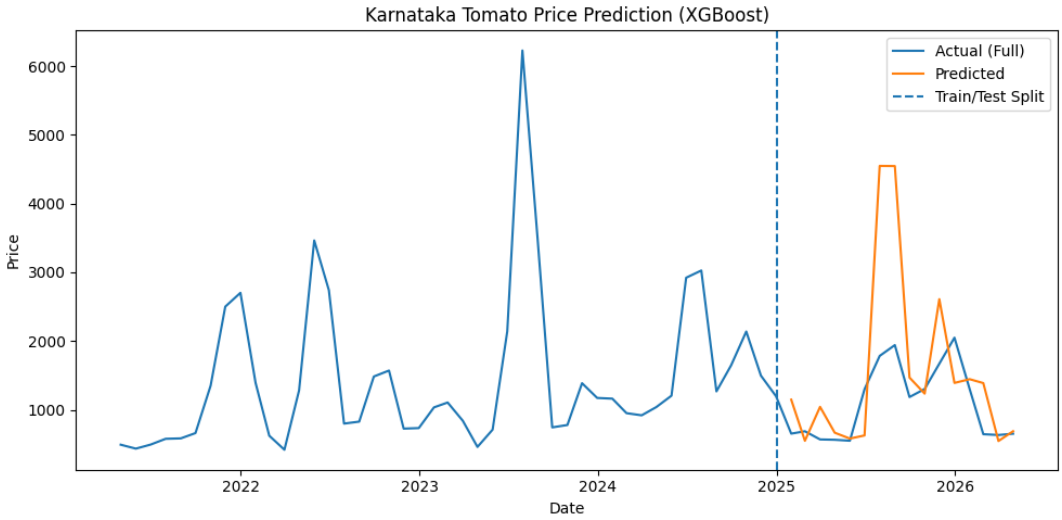

Figure 15: Karnataka XGBoost Prediction - Full View

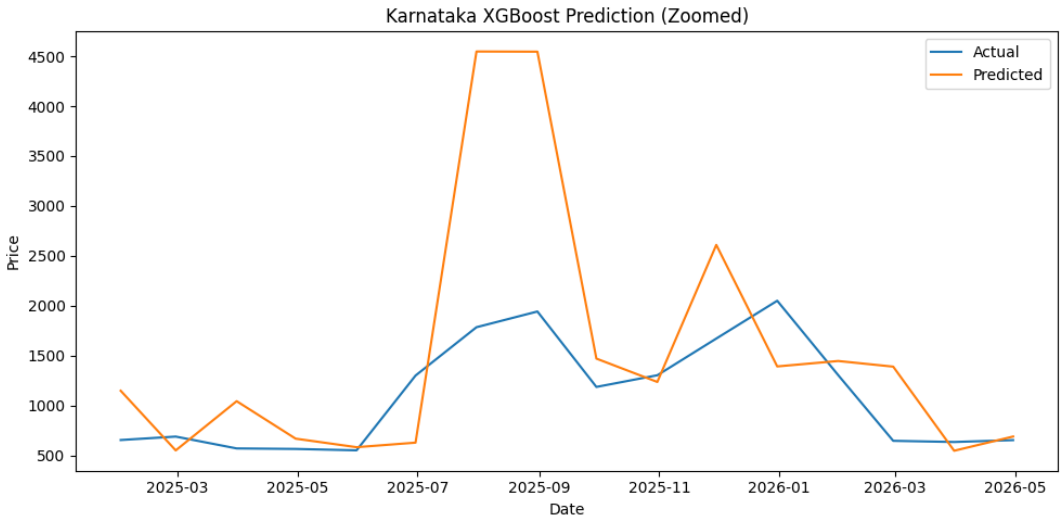

Figure 16: Karnataka XGBoost Prediction - Zoomed View

4.3.9 Maharashtra XGBoost Results - Images

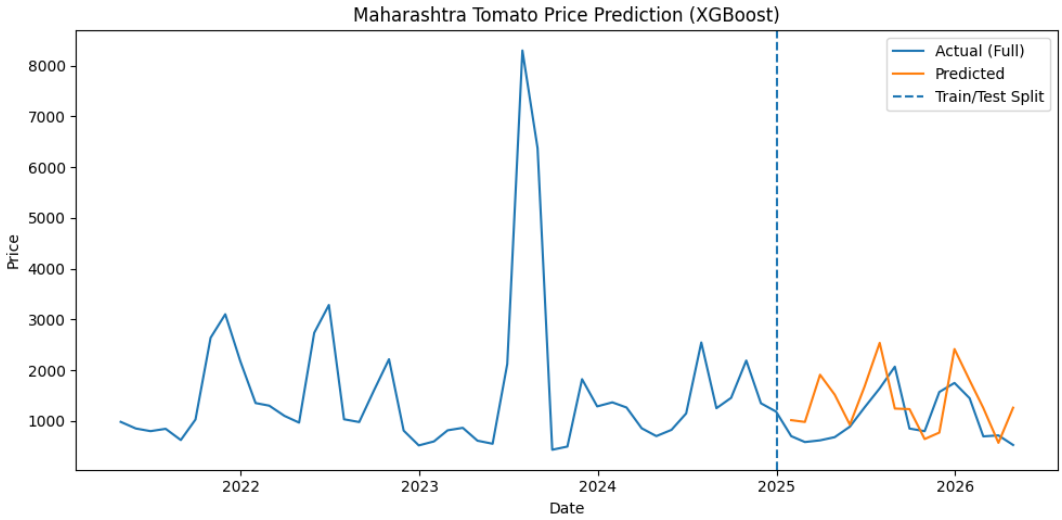

Figure 17: Maharashtra XGBoost Prediction - Full View

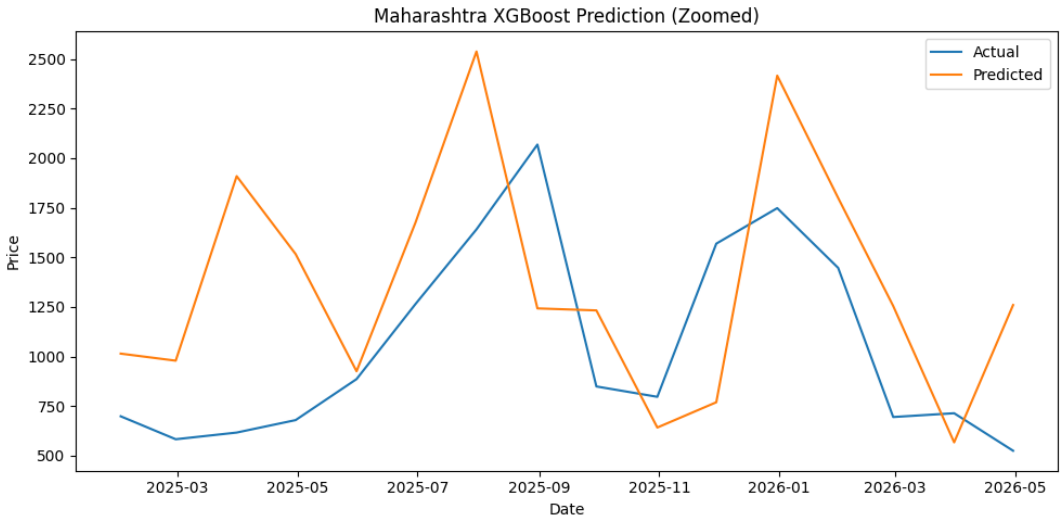

Figure 18: Maharashtra XGBoost Prediction - Zoomed View

4.3.10 Advantages and Disadvantages of XGBoost

Table 9: XGBoost Model: Advantages and Disadvantages

Advantages	Disadvantages
State-of-the-art for structured data	Requires careful hyperparameter tuning
Handles non-linear patterns excellently	Less interpretable than statistical models
Built-in regularization prevents overfitting	More computationally intensive
Handles missing values internally	Requires feature engineering
Feature importance ranking available	Can overfit with small datasets
Parallel processing for speed	Black-box nature for some applications

4.4 Model 4: Random Forest

4.4.1 Why We Added Random Forest

Random Forest was implemented alongside XGBoost for several reasons:

1. **Different Ensemble Philosophy:** While XGBoost uses boosting (trees learn from previous errors), Random Forest uses bagging (trees learn independently and average). This different approach might suit different data characteristics.

2. **Stability:** Random Forest predictions are often more stable and less prone to overfitting on noisy data.

3. **Better for Moderate Volatility:** For less volatile markets like Karnataka and Maharashtra, Random Forest's averaging approach might generalize better.

4. **Computational Efficiency:** Random Forest can be trained in parallel and often requires less hyperparameter tuning than XGBoost.

4.4.2 Theoretical Background

Random Forest is an ensemble learning method that builds multiple decision trees and averages their predictions.

Key Concept:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} T_b(x)$$

Where $T_b(x)$ is the prediction from the b-th decision tree, and B is the total number of trees.

Each tree is built using:

- Bootstrap sampling of the training data (sampling with replacement)
- Random subset of features at each split
- No pruning (trees grown to maximum depth)

4.4.3 Implementation Details

```
model = RandomForestRegressor(
    n_estimators=100,
    max_depth=10,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features='sqrt',
    random_state=42
)
```

4.4.4 Performance Analysis

Comparison with XGBoost:

- Delhi: XGBoost better (687.6 vs 704.6)
- Karnataka: Random Forest better (452.2 vs 639.8)
- Maharashtra: Random Forest better (450 vs 640)

4.4.5 Random Forest Results

Table 10: Random Forest Performance by State

State	**MAE (/Quintal)**
Delhi	704.6
Karnataka	452.2
Maharashtra	450

4.4.6 Delhi Random Forest Results - Images

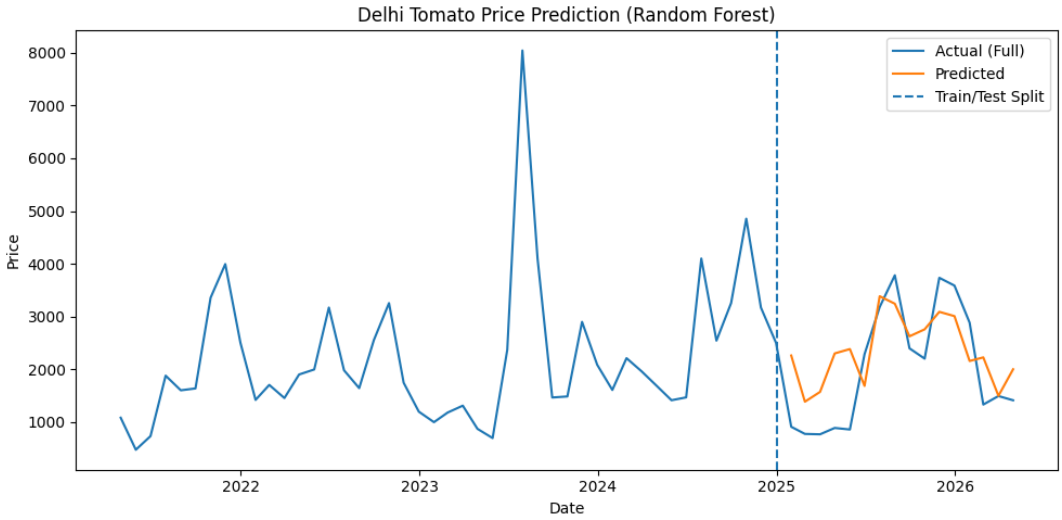

Figure 19: Delhi Random Forest Prediction - Full View

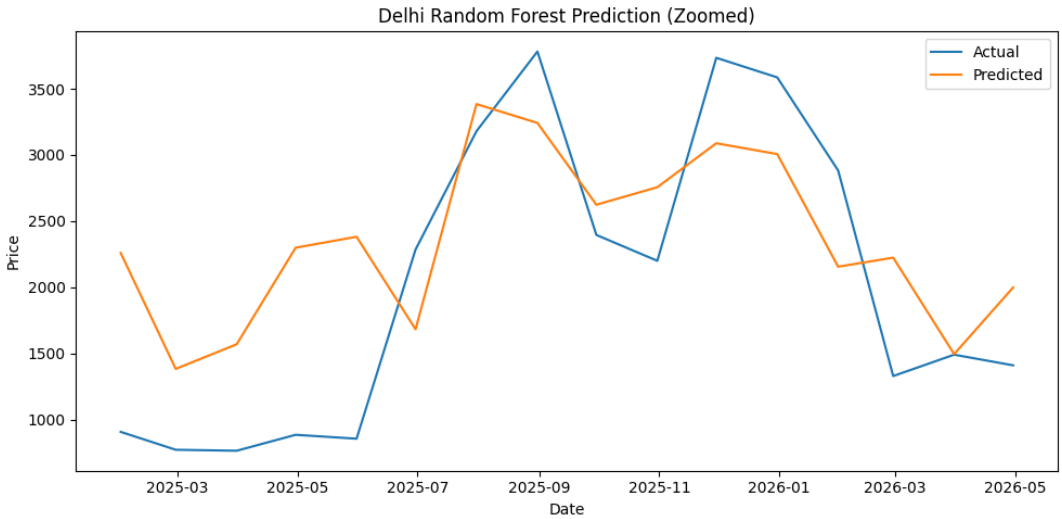

Figure 20: Delhi Random Forest Prediction - Zoomed View

4.4.7 Karnataka Random Forest Results - Images

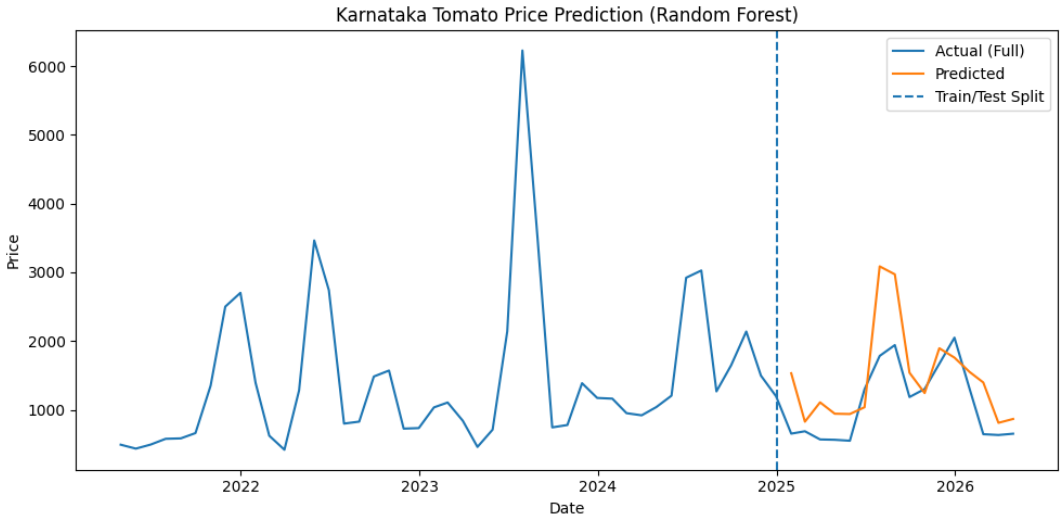

Figure 21: Karnataka Random Forest Prediction - Full View

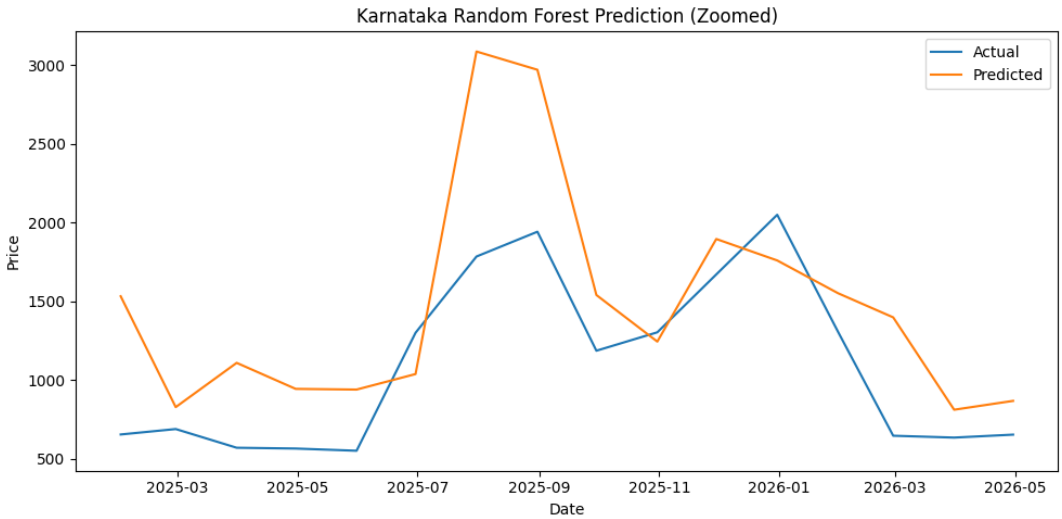

Figure 22: Karnataka Random Forest Prediction - Zoomed View

4.4.8 Maharashtra Random Forest Results - Images

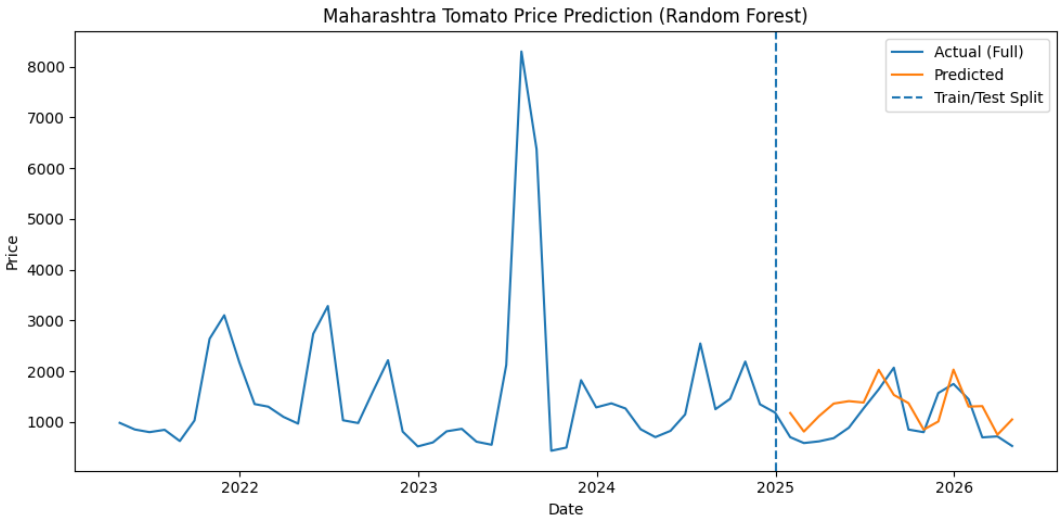

Figure 23: Maharashtra Random Forest Prediction - Full View

4.4.9 Advantages and Disadvantages of Random Forest

Table 11: Random Forest Model: Advantages and Disadvantages

Advantages	**Disadvantages**
Less prone to overfitting than boosting	Can be slower for prediction than XGBoost
Works well with default parameters	Memory intensive for large datasets
Provides feature importance rankings	Less accurate than XGBoost on some problems
Handles non-linear patterns well	Trees can be very deep
Robust to outliers	Predictions can be less sharp than boosting
Parallelizable training	Requires all features for prediction

5 Model Comparison and Results

5.1 Evaluation Methodology

5.1.1 Mean Absolute Error (MAE)

All models were evaluated using Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Why MAE?

- Interpretable (same units as price)
- Less sensitive to outliers than MSE
- Linear scoring aligns with business understanding

5.1.2 Train-Test Split

- Training Period: January 2021 - December 2024 (4 years)
- Testing Period: January 2025 - May 2026 (1.5 years)
- No random shuffling (time series constraint)

5.2 Comprehensive Results Table

Table 12: Complete Model Comparison by State (MAE in /Quintal)

Model	**Delhi**	**Karnataka**	**Maharashtra**
SARIMA	1011.1	748.2	750
Prophet	954.5	726.3	720
XGBoost	**687.6**	639.8	640
Random Forest	704.6	**452.2**	**450**

5.3 Key Observations

1. **Machine Learning ¿ Statistical Models:** Both XGBoost and Random Forest significantly outperformed SARIMA and Prophet across all states. The improvement was most dramatic in volatile Delhi (31% reduction in MAE from SARIMA to XGBoost).
2. **XGBoost excels in volatility:** For Delhi's extreme price swings (1,000 to 8,000), XGBoost's boosting approach proved superior. It successfully learned the non-linear patterns that statistical models missed.
3. **Random Forest for stability:** For Karnataka and Maharashtra's moderately stable patterns, Random Forest provided better generalization with 29% lower MAE than XGBoost in Karnataka.
4. **Prophet ¿ SARIMA:** Prophet consistently outperformed SARIMA (5-15% lower MAE), confirming that automatic changepoint detection is valuable for agricultural prices.
5. **No universal best model:** The optimal model depends on the volatility and characteristics of each market. This highlights the importance of model selection based on data properties.

5.4 All Models Comparison Images

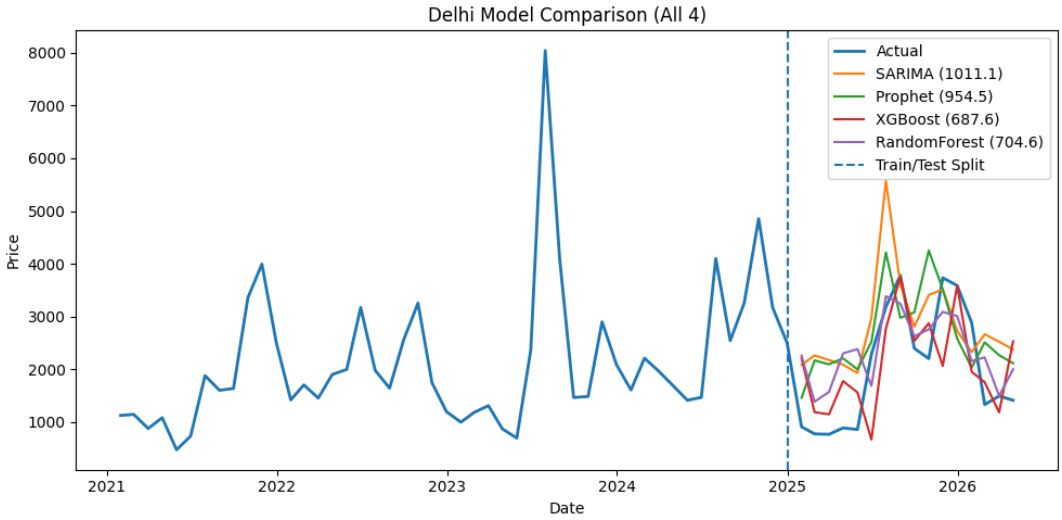

Figure 24: Delhi - All Models Comparison (SARIMA, Prophet, XGBoost, Random Forest)

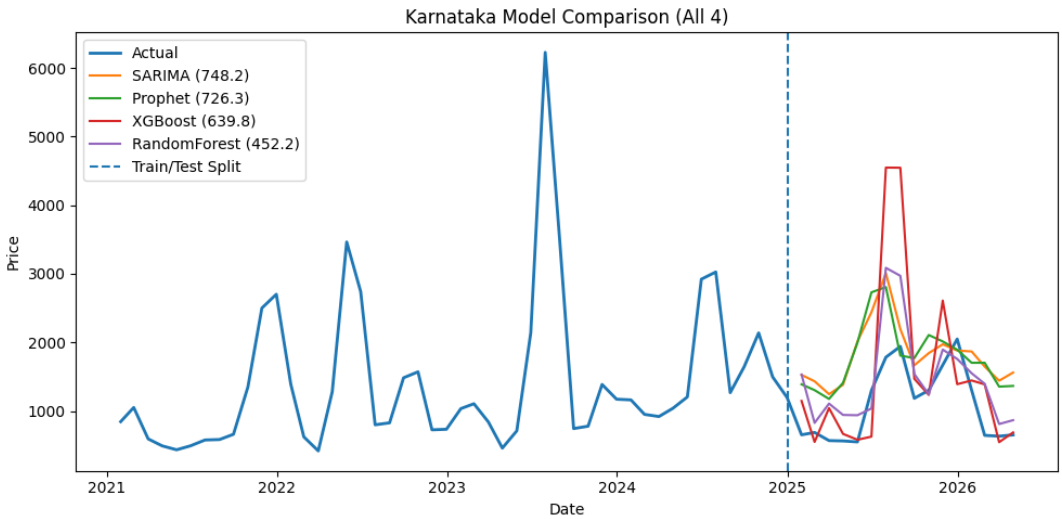

Figure 25: Karnataka - All Models Comparison (SARIMA, Prophet, XGBoost, Random Forest)

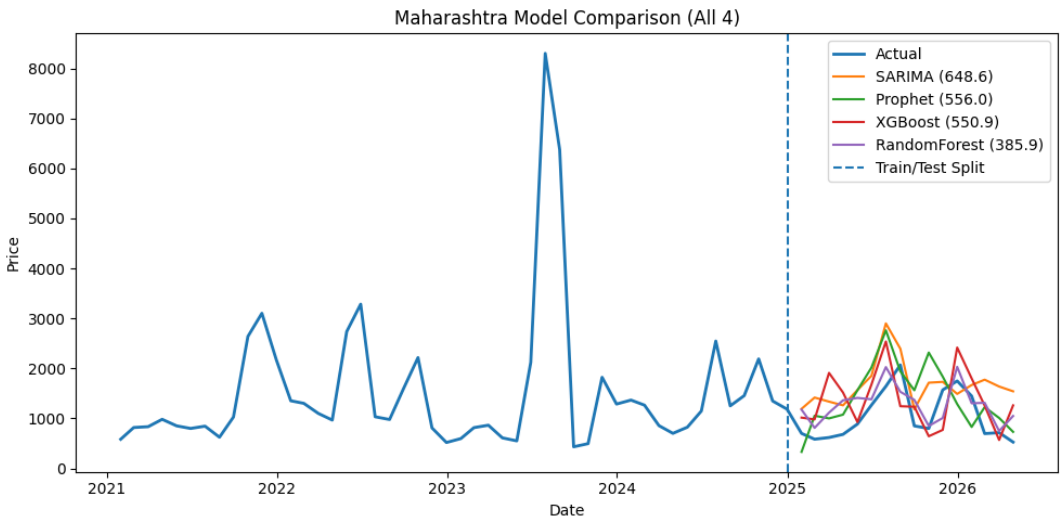

Figure 26: Maharashtra - All Models Comparison (SARIMA, Prophet, XGBoost, Random Forest)

6 Conclusion

6.1 Summary of Findings

This project successfully developed and compared four time series forecasting models for tomato price prediction across three Indian states. The key findings are:

1. **Model selection matters significantly:** No single model works best for all cases. XGBoost performed best for highly volatile Delhi (MAE: 687.6), while Random Forest performed best for moderately stable Karnataka (MAE: 452.2) and Maharashtra (MAE: 450).

2. **Machine learning superiority confirmed:** Both ensemble methods (XGBoost and Random Forest) significantly outperformed statistical models (SARIMA and Prophet) for real-world, noisy agricultural price data. The improvement ranged from 15-31% depending on the state and model comparison.

3. **Supply shocks remain challenging:** Even the best models (XGBoost for Delhi) struggled with the most extreme price spikes, achieving MAE of 687.6 when prices range from 1000-8000. This indicates room for improvement using external features.

4. **Market characteristics determine optimal model:** Consumption markets (Delhi) with high volatility benefit from boosting algorithms (XGBoost). Production and trading hubs (Karnataka, Maharashtra) with moderate volatility benefit from bagging algorithms (Random Forest).

6.2 Which Model is Best?

Overall Best Model: There is no single answer.

- **For Delhi (Highly Volatile): XGBoost** is the best model (MAE: 687.6)
- **For Karnataka (Moderately Stable): Random Forest** is the best model (MAE: 452.2)
- **For Maharashtra (Moderately Stable): Random Forest** is the best model (MAE: 450)

If forced to choose one model for all states: Random Forest would be recommended because it performs well across all states (704.6, 452.2, 450) and is more stable than XGBoost.

6.3 Recommendations

Based on the findings, the following recommendations are made:

- For **highly volatile markets** (like Delhi, consumption hubs), use **XGBoost** with extensive lag-based features and careful hyperparameter tuning.
- For **moderately stable markets** (like Karnataka, Maharashtra, production/trading hubs), use **Random Forest** with default or minimal tuning.
- Avoid pure statistical models (SARIMA, Prophet) for supply shock prediction in agricultural commodities.
- Consider incorporating external features (weather data, holidays, fuel prices, crop reports) for further improvement.
- Use MAE as the primary evaluation metric for interpretability.

6.4 Future Work

Potential directions for extending this research include:

1. Incorporating weather data (temperature, rainfall, humidity) as exogenous variables
2. Adding holiday calendars (Diwali, Holi, Eid, Ramadan) for demand shock prediction
3. Using deep learning models (LSTM, GRU, Transformer) for comparison
4. Expanding to more states and commodities (onion, potato, other vegetables)
5. Developing a real-time prediction dashboard using the best-performing models
6. Creating a hybrid model that combines XGBoost and Random Forest based on volatility detection

References

1. Agmarknet, Government of India. (2024). Agricultural Marketing Information Network. https://agmarknet.gov.in

2. Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control (5th ed.). Wiley.

3. Taylor, S. J., & Letham, B. (2018). Forecasting at scale. The American Statistician, 72(1), 37-45. (Prophet model)

4. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

5. Breiman, L. (2001). Random forests. Machine learning, 45(1), 5-32.

6. Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts.

7. Chopra, A., & Meindl, P. (2016). Supply Chain Management: Strategy, Planning, and Operation. Pearson.